Monitoring meets modularity: a framework for scalable HPC telemetry across diverse sources

Gabriel Nixon Raj [1] Kadidia Konate [2] Wahid Bhimji [2]

[1]New York University [2]Lawrence Berkeley National Laboratory

Abstract

This project presents a modular pipeline for generating weekly reports that bring together system activity from across the HPC environment. It collects structured data from multiple sources and produces summaries that give a comprehensive view of system usage and behavior. The pipeline supports consistent ingestion, Parquet-based storage, and report generation, making it easy to adapt as new data sources are added. The goal is to help operations and support teams move from scattered information to a unified understanding of how the system is being used over time.

Background & Motivation

Perlmutter generates data from job schedulers, support systems, and communication tools—but that data is often siloed. Different teams report from their own sources, and stitching everything together each week means extra coordination and duplicated effort. This project builds a unified, automated reporting system to reduce overhead, save time, and give teams a clearer view of how the system is running. It correlates jobs, downtime, support activity, and informal reports to improve situational awareness. The results feed directly into reports submitted to the Department of Energy, so accuracy and consistency are key.

Why These Data Sources

To support weekly reporting and match operational metrics, this project pulls from five key data sources. Each one enables tracking specific signals, such as Perlmutter's downtime, queue wait times, and GPU capability workloads.

- **SLURM (`sacct`, `squeue`):** Used to calculate job counts, queue wait times (Regular QOS), partition and node usage, GPU capability job detection, job states, CPUHours, and system utilization.
- **ServiceNow:** Captures both scheduled and unscheduled downtime events. Also tracks support ticket timing, open backlog, and 3-day resolution metrics to align with monthly support accountability goals.
- **Gmail:** Provides timestamps and metadata for formal ticket and report conversations.
- **Slack:** Surfaces real-time operational context, including data from both slack bots and certain communication channels.
- **Outage Logs:** Parsed from official NERSC maintenance reports and availability summaries, these provide structured details on both scheduled and unscheduled outages. They are essential for calculating metrics like MTTF/MTTI and availability percentages.

Together, these sources allow the reporting system to reconstruct the structure of the report. The system is also designed to be modular and scalable, so new data sources can be added as needed without redesigning the core reporting workflow.

System Design

The system uses a modular design to pull in data from multiple platforms and generate unified weekly reports. Each data source—like job logs from Slurm, support records from ServiceNow, or internal communication via Slack and Gmail—is handled by a dedicated collector script.

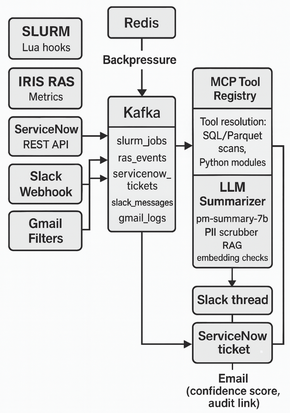
Collected data is streamed through Apache Kafka, which buffers and transports the records into a Delta Lake-backed storage layer. The data is stored in Parquet format, partitioned by source and time, making it easier to query and summarize.

This structure helps reduce manual overhead while keeping reporting consistent, scalable, and aligned across teams. For reporting, a lightweight model scans the stored data weekly and assembles high-level summaries, such as job volumes, wait times, outage impacts, and ticket trends, into structured reports for internal use.

1. **Parquet for Storage Efficiency:** Columnar storage with Parquet allows fast filtering and aggregation across large, timestamped datasets—ideal for weekly job and support trends.
2. **Kafka + Delta Lake:** Kafka handles high-throughput, decoupled ingestion from each source, while Delta Lake offers versioned, scalable storage and ACID guarantees for downstream reporting.
3. **Simple Report Model:** The reporting layer is intentionally lightweight—built to extract weekly signals without requiring heavyweight dashboards or database integration.

Architecture and Expansion

The architecture has been deliberately designed to stay open and modular. Several components have been considered for future integration but are not yet activated. Keeping these design hooks in place allows the system to evolve without needing disruptive rewrites.



System Architecture

Two additional components strengthen the pipeline's long-term viability:

- **Redis:** A lightweight buffer that absorbs backpressure during data spikes, ensuring no messages are lost when Kafka slows down.
- **MCP Tool Registry:** A modular dispatcher that assigns tools to data types, making the system easy to scale and extend with minimal code changes.

Current Progress and Implementation Status

The core reporting pipeline is up and running, with full integration of SLURM job data via `sacct` and `squeue`. These are streamed through the architecture described above, producing weekly summaries that answer key questions around job counts, CPU hours, queue delays, and utilization trends. In parallel, a prototype Slack bot has been developed to extract operational insights from messaging channels. While this pipeline currently runs on sample data, early results are promising. We plan to connect it to real Perlmutter communication channels like `#utilization` and the PM queue slack bot to surface internal discussion around job states, system events, and informal incident tracking. Work on ServiceNow and Gmail ingestion is still in the planning stage. The architecture is built to support them, but full pipelines for those sources have not yet been implemented.

```
Weekly HPC Job Summary
Timeline – 2025-07-21T20:51:06.485847

- Total Jobs: 25814154
- COMPLETED: 16953650 jobs
- FAILED: 1947993 jobs
- NODE_FAIL: 12099 jobs
- OUT_OF_MEMORY: 57087 jobs
- PREEMPTED: 3568 jobs
- REQUEUED: 150984 jobs
- RUNNING: 2528668 jobs
- State: 2016 jobs
- TIMEOUT: 1239280 jobs
- Users Active: 2141
```

Conclusion

This project demonstrates a practical approach to centralizing operational insights from Perlmutter's diverse data streams. While initial progress has focused on SLURM and Slack data, the system has been designed with flexibility in mind. Future stages will bring in additional sources like ServiceNow and Gmail. The next data source that we are planning on incorporating is outage logs. Even in its early stages, this project shows that thoughtful data integration, done incrementally and openly, can offer real impact in high-performance computing environments.

Acknowledgement

This work was completed as part of the Summer 2025 Internship at Lawrence Berkeley National Laboratory. I would like to thank my mentor, Kadidia Konate, for her support and guidance throughout this project. I also acknowledge the support of the DOE and the Computing Sciences Area at LBNL for providing the resources and environment that made this work possible.

gr2513@nyu.edu